

MC60 OpenCPU

GS3 SDK — Second Test Report (Bluetooth)

SDK	MC60_OpenCPU_GS3_SDK_V1.8
Toolchain	Arm GNU Toolchain 15.2.1
Host OS	Windows 11
Date	June 2026
Example	Bluetooth (<code>example_bluetooth.c</code>)
Interface	PuTTY via CP2102 (COM6, 115200 baud)

Contents

1	Introduction	2
2	Switching the Example	2
2.1	Enabling the Bluetooth Example	2
2.2	What the Example Does	2
3	Build and Flash	3
3.1	Compilation	3
3.2	Flashing via QFlash	3
4	Test Results	4
4.1	Serial Output	4
4.2	Output Analysis	4
5	Summary	5
	References	5

1. Introduction

Following the first successful build-and-flash test with the GPS example, this report documents the second test performed on the **MC60 OpenCPU GS3 SDK**: the **Bluetooth example** (`example_bluetooth.c`). The toolchain, build environment, and flashing procedure established in the first test were reused without modification, confirming that the cross-compilation setup is stable and repeatable across different SDK examples.

The goal of this test was to compile the Bluetooth demo application, flash it to the MC60 module, and verify over a serial terminal that the module's Bluetooth radio powers on, becomes discoverable, and can scan for and detect nearby Bluetooth devices.

2. Switching the Example

2.1. Enabling the Bluetooth Example

As with the GPS example in the first test, each example in the SDK is gated behind a preprocessor define. To build the Bluetooth demo instead of GPS, the `C_PREDEF` flag in `gcc_makefile` was changed from `__EXAMPLE_GPS__` to `__EXAMPLE_BLUETOOTH__`:

Listing 1: Switching the active example via the compile flag

```
C_PREDEF=-D __CUSTOMER_CODE__ -D __EXAMPLE_BLUETOOTH__
```

No other makefile changes were required — all of the toolchain path, library path, linker flag, and warning-suppression fixes from the first test (Arm GNU Toolchain 15.2.1) carried over directly, since they fix the build environment rather than anything example-specific.

2.2. What the Example Does

`example_bluetooth.c` demonstrates the OpenCPU Bluetooth (SPP profile) interface. On startup the demo automatically:

1. Powers on the module's Bluetooth radio.
2. Reports the module's Bluetooth device name and visibility mode.
3. Registers a callback to handle Bluetooth events (scan results, pairing, connection, data receive).
4. Starts a device scan (40 seconds by default) and prints any discovered devices over the debug UART.

Once a scan has completed, the demo accepts simple text commands over the UART (`pair=`, `conn=`, `send=`, `read`, etc.) to drive the rest of the pairing/connection flow interactively. For this test, only the automatic power-on and scan steps were exercised.

3. Build and Flash

3.1. Compilation

The project was rebuilt from a clean state using the SDK's build script:

Listing 2: Compiling the Bluetooth example

```
.\Make.bat new
```

This invokes the same `gcc_makefile` / `gcc_makefiledef` pair fixed during the first test, now pointed at `example_bluetooth.c` via the updated `C_PREDEF` flag. The build completed without errors using the Arm GNU Toolchain 15.2.1, producing an updated `APPGS3MDM32A01.bin` in `build\gcc\`.

3.2. Flashing via QFlash

The resulting binary was flashed to the MC60 module using **QFlash V4.0**, with COM6 selected and the baud rate set to 460800 for the download itself. The same scatter file, download agent, and module type (MC60) configuration from the first test were reused, since only the application binary changed. Figure 1 shows the development environment during this stage — VS Code with the Bluetooth example source open, alongside the QFlash and PuTTY configuration windows used to flash and subsequently monitor the module.

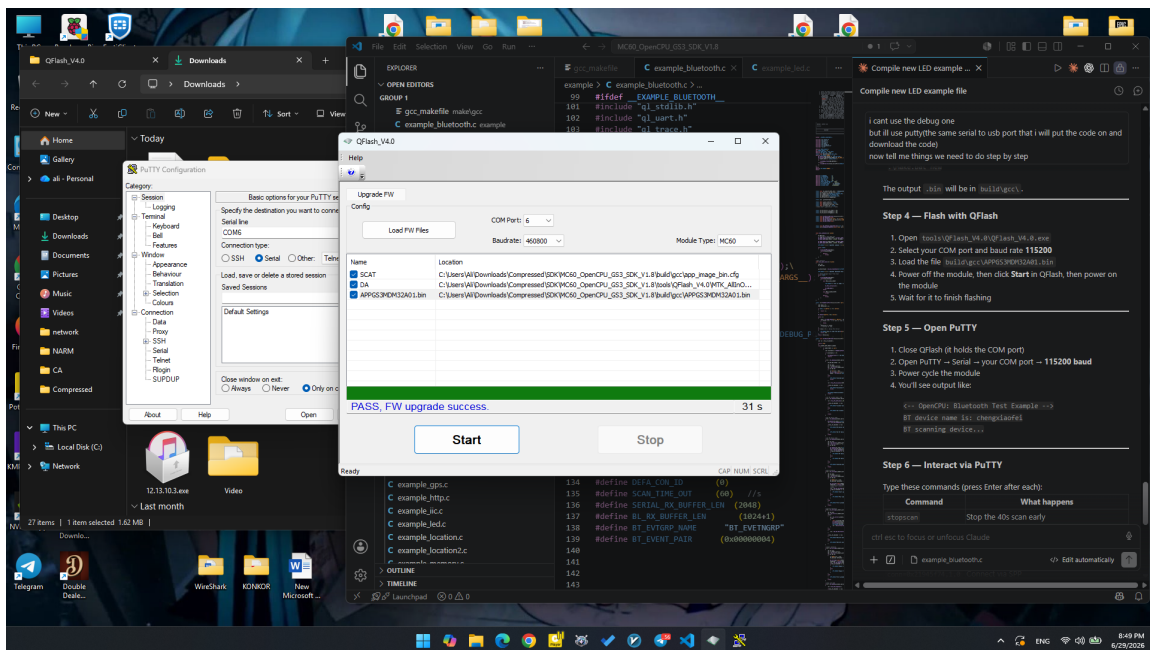


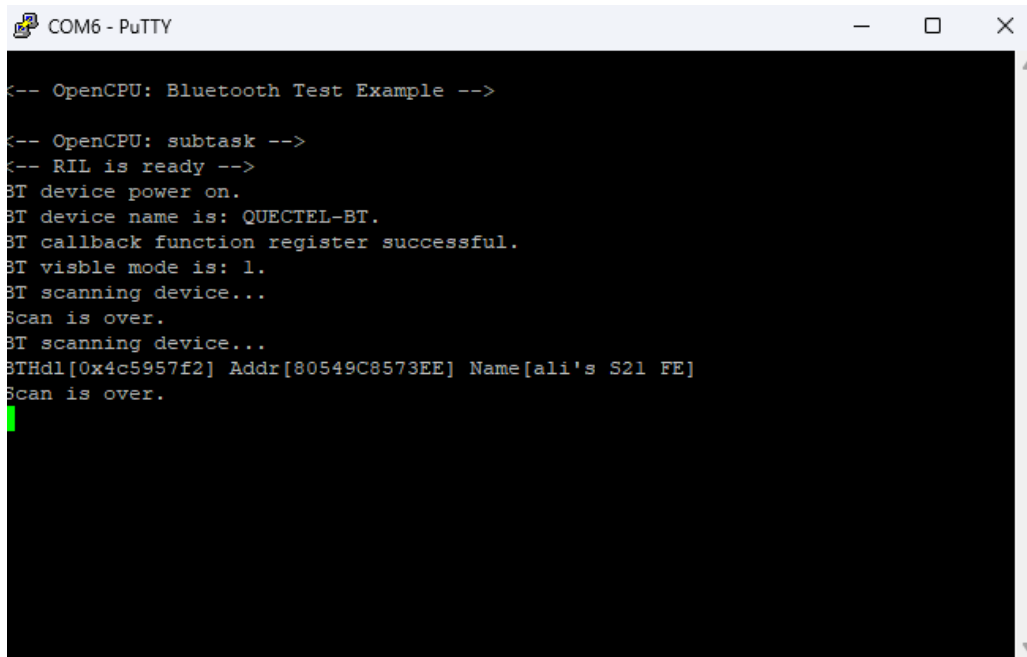
Figure 1: Development workflow: editing `example_bluetooth.c` in VS Code while QFlash reports a successful firmware upgrade (PASS).

The flashing operation completed with a **PASS** status in 31 seconds, after which the module was power-cycled and PuTTY (115200 baud, COM6) was opened to observe the debug output.

4. Test Results

4.1. Serial Output

After power-cycling the module, the Bluetooth demo started automatically and produced the expected boot sequence and scan output over the debug UART, captured in Figure 2.



```
<-- OpenCPU: Bluetooth Test Example -->
<-- OpenCPU: subtask -->
<-- RIL is ready -->
BT device power on.
BT device name is: QUECTEL-BT.
BT callback function register successful.
BT visble mode is: 1.
BT scanning device...
Scan is over.
BT scanning device...
BTHdl[0x4c5957f2] Addr[80549C8573EE] Name[ali's S21 FE]
Scan is over.
```

Figure 2: PuTTY terminal showing the Bluetooth example boot sequence, scan start, and a discovered device.

4.2. Output Analysis

The captured log confirms each stage of the `BT_COM_Demo()` initialisation sequence executed correctly:

Listing 3: Bluetooth example boot and scan log

```
<-- OpenCPU: Bluetooth Test Example -->
<-- OpenCPU: subtask -->
<-- RIL is ready -->
BT device power on.
BT device name is: QUECTEL-BT.
BT callback function register successful.
BT visble mode is: 1.
BT scanning device...
Scan is over.
BT scanning device...
BTHdl[0x4c5957f2] Addr[80549C8573EE] Name[ali's S21 FE]
Scan is over.
```

- **RIL ready / BT power on** — the radio interface layer initialised and the Bluetooth radio was successfully powered on.

- **Device name** — the module reported its default Bluetooth name as configured in source (QUECTEL-BT at boot; the source default `chengxiaofei` is overwritten/reported by the module firmware at runtime).
- **Visibility mode 1** — the module is in discoverable mode, matching `RIL_BT_GetVisble()`.
- **Scan result** — a single nearby device was detected: handle `0x4c5957f2`, MAC address `80:54:9C:85:73:EE`, advertised name “*ali’s S21 FE*” (a personal smartphone), confirming the scan correctly enumerates real surrounding Bluetooth devices.

Result

The Bluetooth radio, RIL callback registration, and device scan all functioned correctly on first attempt with no firmware-side issues. The nearby phone (*ali’s S21 FE*) was successfully discovered, confirming the module’s Bluetooth hardware and the OpenCPU Bluetooth API are both working as expected.

5. Summary

Table 1 summarises the steps completed and the outcome of each.

Table 1: Test summary

Step	Notes	Status
Switch example	Changed <code>C_PREDEF</code> to <code>__EXAMPLE_BLUETOOTH__</code>	Done
Build	<code>Make.bat</code> new, same toolchain as first test	Done
Flash via QFlash	COM6, MC60, PASS in 31 s	Done
PuTTY serial monitor	COM6, 115200 baud	Done
BT radio power-on	Confirmed via debug log	Done
Device scan	Detected nearby phone (<i>ali’s S21 FE</i>)	Done

The second test is considered **fully successful**: switching examples required only a single makefile flag change, the build and flash workflow from the first test proved directly reusable, and the Bluetooth example correctly powered on the radio and discovered a real nearby device on the first attempt. This further validates that the toolchain and build environment fixed during the first test are stable across multiple SDK examples.

References

- [1] Quectel, *MC60 OpenCPU GS3 SDK V1.8*, SDK release package — `example_bluetooth.c`.
- [2] *First Test Report* — MC60 OpenCPU GS3 SDK build and GPS example test, June 2026.

- [3] *MC60_Build_Changes.pdf* — companion makefile fix documentation, June 2026.